

Linux pour l'admin cloud

Jour 1 — Prendre la main sur la machine

Bloc CL-LINUX (jours 6 à 9 du cursus) — le terminal devient votre poste de travail

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Expliquer **pourquoi Linux est l'OS du cloud** et distinguer **noyau, distribution, shell**.
- **Lancer, arrêter et supprimer** une VM Ubuntu Server 24.04 LTS avec Multipass (ou VirtualBox).
- **Naviguer** dans l'arborescence (`pwd`, `ls`, `cd`) et **manipuler** fichiers et répertoires (`cp`, `mv`, `rm`, `mkdir`).
- **Lire** un fichier avec `cat`, `less`, `head`, `tail` et **trouver de l'aide** avec `man` et `--help`.
- **Éditer** un fichier avec nano, et **sortir vivant** de vim.
- Créer des **utilisateurs et des groupes**, utiliser **sudo** à bon escient.
- Lire et modifier des **permissions** (`rwX`, notation octale, `chmod`, `chown`, `umask`).

À partir d'aujourd'hui et jusqu'à la fin du cursus, **tout se passe dans ce terminal** :
c'est exactement celui que vous aurez en SSH sur une instance EC2.

Plan de la journée

1. **Pourquoi Linux dans le cloud ?** – l'OS invisible qui fait tourner Internet.
2. **Distribution, noyau, shell** – et lancement de votre VM (💻 démo 1-1).
3. **Naviguer et manipuler** – pwd/ls/cd, cp/mv/rm/mkdir, cat/less/head/tail (✏️ exercice 1-1).
4. **Trouver de l'aide + éditeurs** – man, --help ; nano, vim de survie.
5. **Utilisateurs, groupes, sudo** – /etc/passwd, adduser, su, sudo.
6. **Permissions** – rwx, notation octale, chmod/chown, umask (✏️ exercice 1-2).
7. Récap + quiz de fin de journée.

1. Pourquoi Linux dans le cloud

L'OS que vous ne voyez jamais fait tourner tout ce que vous voyez

- **Plus de 90 % des instances des clouds publics** tournent sous Linux ; même chez Microsoft, **la majorité des VM Azure** sont des VM Linux.
- **Toutes** les images de base des fournisseurs cloud sont disponibles en Linux — et l'**AMI Ubuntu 24.04 LTS est l'une des images les plus déployées sur AWS**.
- Les 500 plus gros supercalculateurs, Android, les box Internet, Kubernetes, Docker : Linux partout.

Et surtout : **un serveur n'a ni écran, ni souris, ni bureau graphique.**

Votre seul accès à une EC2 sera une session SSH : un terminal, un shell, du texte.

Fil conducteur du cursus : automatiser, superviser, sécuriser.

Les trois passent par la ligne de commande — on ne clique pas dans un serveur.

Où ce bloc vous emmène dans le cursus

Ce que vous apprenez ces 4 jours	Où vous le réutilisez
J1 : navigation, permissions, sudo	Chaque session SSH jusqu'à la fin du cursus
J2 : processus, services systemd, APT, journaux	Déploiement StockLine, diagnostic de pannes
J3 : scripts bash, cron/timers, disques, réseau local	Automatisation, CL-AWS1 (EBS, user data)
J4 : SSH, durcissement, déploiement StockLine	CL-TP1 (déploiement noté), toute votre carrière

Dès **CL-TP1** (jour 14), vous déploierez StockLine sur une VM **sans filet** :

tout ce qui suit est de l'entraînement pour ce moment-là.

La VM que vous créez ce matin **vit toute la semaine** : vous la retrouverez demain avec vos fichiers, comme une vraie machine de production.

2. Distribution, noyau, shell

Et votre première machine virtuelle

Anatomie d'une distribution Linux

```
+-----+
|          DISTRIBUTION  (ex. Ubuntu)          |
| +-----+ |
| | Applications : nginx, python3, htop... | |
| +-----+ |
| | Gestionnaire de paquets : APT (.deb) | |
| +-----+ |
| | Shell + outils GNU : bash, ls, cp, grep... | |
| +-----+ |
| | NOYAU LINUX : matériel, processus, | |
| | mémoire, réseau, systèmes de fichiers | |
| +-----+ |
+-----+
```

Noyau = le cœur, commun à tous. **Distribution** = noyau + outils

- gestionnaire de paquets + choix d'intégration.

Les grandes familles – et notre choix : Ubuntu Server 24.04 LTS

Famille	Distributions	Paquets	Où vous la croiserez
Debian	Debian, Ubuntu , Mint	APT (<code>.deb</code>)	La plupart des VM cloud, ce cursus
Red Hat	RHEL, Rocky, Fedora, Amazon Linux	DNF (<code>.rpm</code>)	Entreprises, AMI Amazon Linux
Autres	Alpine (conteneurs), Arch, SUSE	divers	CL-CONT (images Docker)

Notre référence pour tout le cursus : **Ubuntu Server 24.04 LTS** (« Noble Numbat »).

- **LTS** = Long Term Support : 5 ans de mises à jour de sécurité – c'est ce qu'on déploie en production, jamais une version intermédiaire.
- **Server** = sans interface graphique : comme dans le cloud.
- Noyau 6.8, image officielle disponible sur AWS, Azure et GCP : **les commandes d'aujourd'hui sont valables telles quelles sur vos futures instances.**

Le shell : votre interlocuteur

Le **shell** lit vos commandes, les exécute, affiche le résultat. Le nôtre : **bash**.

```
ubuntu@srv-linux:~$ ls -l /var/log
|         |         |         |         |
|         |         |         |         +-- argument (sur quoi ?)
|         |         |         +----- option (comment ?)
|         |         +----- commande (quoi ?)
|         +----- $ = utilisateur normal (# = root)
|         +----- ~ = répertoire courant (votre HOME)
+----- utilisateur @ machine
```

- Une commande = **commande [options] [arguments]**. Options courtes **-l**, longues **--all**, cumulables : **ls -la**.
- Le prompt vous dit toujours **qui vous êtes, où vous êtes** : réflexe à prendre avant chaque commande sensible — surtout quand vous aurez 4 sessions SSH ouvertes.

Multipass : une VM Ubuntu en 2 minutes (voie principale)

Multipass (éditeur : Canonical, gratuit, macOS/Windows/Linux) télécharge l'image officielle Ubuntu et lance une VM en une commande :

```
multipass launch 24.04 --name srv-linux --cpus 2 --memory 4G --disk 20G
multipass list           # les VM et leur état
multipass info srv-linux # détails, dont l'IP de la VM
multipass shell srv-linux # ouvrir un shell DANS la VM
```

Cycle de vie — la VM garde ses fichiers entre deux démarrages :

```
multipass stop srv-linux      # extinction propre (fin de journée)
multipass start srv-linux     # redémarrage (demain matin)
multipass delete srv-linux && multipass purge # destruction définitive
```

Utilisateur par défaut : **ubuntu**, sudo sans mot de passe —
exactement comme sur une instance EC2 Ubuntu.

Votre poste, la VM : qui est qui ?

```

VOTRE POSTE ("l'hôte")                                macOS / Windows / Linux
+-----+-----+
| terminal de l'hôte      Multipass (hyperviseur)      |
|      |                  |                            |
|      | multipass shell    v                            |
|      +-----> +-----+                            |
|                  | VM "srv-linux" |                  |
|                  | Ubuntu Server 24.04 |              |
|                  | 2 vCPU / 4 Go / 20 Go |             |
|                  | utilisateur: ubuntu |             |
|                  +-----+                            |
+-----+-----+
  
```

Sur AWS, même schéma : la VM s'appelle **instance EC2**, l'hyperviseur appartient à Amazon, et `multipass shell` devient `ssh`.

Tout ce qu'on casse dans la VM reste dans la VM.

Plan B : VirtualBox (si Multipass est impossible sur votre poste)

Si Multipass refuse de démarrer (virtualisation imbriquée, politique de sécurité du poste), on obtient **la même VM** avec VirtualBox :

1. Télécharger l'**ISO Ubuntu Server 24.04 LTS** (ubuntu.com/download/server).
2. Créer une VM : **2 vCPU, 4 Go de RAM, 20 Go de disque**.
3. Dérouler l'installateur : utilisateur `ubuntu`, cocher « **Install OpenSSH server** ».
4. Réseau : « **Accès par pont** » (la VM a une IP sur votre réseau) — ou NAT + redirection de port **hôte 2222** → **invité 22** si le pont est interdit.

Différences avec Multipass au quotidien :

- connexion : `ssh ubuntu@<IP-VM>` (ou `ssh -p 2222 ubuntu@127.0.0.1` en NAT) ;
- `sudo` demande votre mot de passe (défini à l'installation).

Tout le reste du bloc est identique quelle que soit la voie choisie.

Démo 1-1 — Première VM avec Multipass

(25 min)

Fichier : `demos/03-cl-linux/demo-1-1-premiere-vm-multipass.md`

En direct, de zéro jusqu'au shell :

- `multipass find`, `launch`, `list`, `info`, `shell` — la VM naît sous vos yeux ;
- tour du propriétaire : `whoami`, `hostnamectl`, `lsb_release -a`, `uname -r` ;
- premiers pas : `ls`, `cd`, lecture du prompt ;
- `multipass stop` / `start` : la VM garde tout ;
- lancement avec `code/03-cl-linux/cloud-init.yaml` : la VM s'auto-configure au premier démarrage (paquets, message d'accueil).

Parallèle AWS à retenir : `multipass launch` = **lancer une instance**,

cloud-init = le « **user data** » d'EC2 — le même fichier YAML fonctionne sur AWS.

Vous refaites tout immédiatement après : **personne ne commence l'exercice 1-1 sans un shell ouvert dans sa VM.**

3. Naviguer et manipuler

L'arborescence, vos mains sur le clavier

L'arborescence Linux : tout part de /

```
/
├─ bin -> usr/bin    la racine : TOUT est dessous
├─ etc               commandes (ls, cp, bash...)
├─ etc               configuration du système
├─ home              répertoires des utilisateurs
│   └─ ubuntu        votre HOME, alias ~
├─ opt               applis tierces (StockLine en J4)
├─ root              HOME de root (pas /home/root)
├─ srv               données servies (partages, web)
├─ tmp               temporaire, purgé au reboot
├─ usr               programmes et bibliothèques
└─ var               données qui grossissent
    ├── log           les journaux (J2)
    └─ backups        les sauvegardes (J3)
```

Pas de `C:\` : les disques se **montent** dans l'arbre (J3).

Se repérer et se déplacer : pwd, cd

```
pwd          # où suis-je ? -> /home/ubuntu
cd /var/log  # chemin ABSOLU : commence par /
cd apt       # chemin RELATIF : depuis le répertoire courant
cd ..        # remonter d'un cran (répertoire parent)
cd           # sans argument : retour au HOME
cd -         # revenir au répertoire précédent
```

Symbole	Signifie
/	la racine (en tête de chemin) ou le séparateur
.	le répertoire courant
..	le répertoire parent
~	votre HOME (/home/ubuntu)

Règle mentale : **absolu** = **adresse postale complète**, **relatif** = « à partir d'ici ».

Dans un script (J3), préférez l'absolu : il marche d'où qu'on le lance.

Lister : ls et ses options qui servent vraiment

```
ls                # le contenu du répertoire courant
ls -l /etc        # format long : droits, propriétaire, taille, date
ls -a ~           # tout, y compris les fichiers cachés (.bashrc, .ssh)
ls -lh /var/log   # tailles lisibles (K, M, G)
ls -lt /var/log   # tri par date, le plus récent en premier
ls -ltr /var/log  # ordre inverse : le plus récent EN BAS (pratique)
```

Lire une ligne de `ls -l` (on décortique les droits en section 6) :

```
-rw-r--r-- 1 root root 1183 Apr 22 10:15 hosts
droits    liens propr. groupe taille date  nom
```

Réflexe d'admin sur serveur : `ls -ltr` dans `/var/log` = « quel journal vient de bouger ? » — premier geste de diagnostic sur une EC2 qui se comporte mal.

Créer, copier, déplacer, supprimer

```
mkdir rapports           # créer un répertoire
mkdir -p projets/2026/logs # -p : crée toute la chaîne d'un coup
cp note.txt note-copie.txt # copier un fichier
cp -r rapports/ archives/ # -r : copier un répertoire (Récursif)
mv note.txt rapports/     # déplacer...
mv rapports/note.txt bilan.txt # ... ou RENOMMER (même commande)
rm bilan.txt              # supprimer un fichier
rm -r archives/           # supprimer un répertoire et TOUT son contenu
```

⚠ **Il n'y a pas de corbeille.** `rm` est immédiat et définitif ; `rm -r` emporte récursivement tout ce qu'il y a dessous, sans confirmation.

Sur un serveur cloud, **aucune restauration magique** : un `rm -r` mal placé sur `/etc` ou `/var` = on recrée l'instance. D'où les sauvegardes (J3).

Filet de sécurité en phase d'apprentissage : `rm -ri archives/` (`-i` = confirmer chaque suppression), et **relire le prompt avant d'appuyer sur Entrée**.

Lire un fichier : cat, less, head, tail

```
cat /etc/hostname           # afficher tout (fichiers COURTS)
less /var/log/syslog        # lire page par page (fichiers longs)
head -n 5 /etc/passwd       # les 5 premières lignes
tail -n 20 /var/log/syslog   # les 20 dernières lignes
tail -f /var/log/syslog      # suivre le fichier EN DIRECT (Ctrl+C pour sortir)
```

Dans **less** : **Espace** page suivante, **b** page précédente, **/motif** rechercher, **n** occurrence suivante, **G** fin du fichier, **q** pour quitter.

Besoin	Outil
Fichier de 3 lignes	cat
Journal de 50 000 lignes	less
« Que vient-il de se passer ? »	tail -n 50
Surveiller en temps réel	tail -f

tail -f sur les journaux : **le geste n°1 de la supervision** – vous le referez en J2 avec **journalctl -f**, puis dans CloudWatch côté AWS.

Travailler plus vite : complétion et historique

La touche Tab est votre meilleure alliée :

```
ls /var/lo<Tab>      # complète en /var/log/  
cd /etc/sys<Tab><Tab> # deux Tab : liste les possibilités (sysctl.d/, systemd/...)
```

- Complète commandes, chemins, noms de fichiers → **zéro faute de frappe**.
- Si rien ne se complète, c'est que le chemin n'existe pas : Tab **vérifie** avant vous.

L'historique — bash retient tout :

```
history          # les commandes numérotées de la session  
history | tail -n 20 # les 20 dernières
```

- **↑** / **↓** : rejouer les commandes précédentes ;
- **Ctrl+R** puis quelques lettres : **recherche interactive** dans l'historique (encore **Ctrl+R** pour remonter, **Entrée** pour exécuter) ;
- **!!** : re-exécuter la dernière commande — le classique **sudo !!** après un « Permission denied ».



Exercice 1-1 — Exploration et manipulation de l'arborescence

Fichier : `exercices/03-cl-linux/exercice-1-1-exploration-arborescence.md`

Durée : 45 min — difficulté 1/5 — seul, dans votre VM

Mise en situation : vous prenez en main un serveur fraîchement livré.

- Construire l'arborescence `~/exploitation` (`scripts/`, `rapports/`, `archives/`) avec `mkdir -p`.
- Copier, déplacer, renommer des fichiers entre ces répertoires.
- Explorer `/var/log` avec `less`, `head`, `tail` — sans rien modifier.
- Trouver une option de commande **par vous-même** avec `man` (avant-goût de la section suivante).

Objectif : que `cd`, `ls`, `cp`, `mv` et Tab deviennent des réflexes — vous n'y penserez plus jamais, comme le changement de vitesse en voiture.

4. Trouver de l'aide, éditer un fichier

man, --help, nano, vim de survie

L'aide est déjà sur la machine : man et --help

```
man ls           # le manuel complet de ls (navigation = celle de less)
ls --help        # l'aide courte : rappel des options, sans quitter le terminal
man -k copy      # chercher dans TOUS les manuels par mot-clé (= apropos)
```

Le manuel est découpé en **sections** – utile quand un nom existe en double :

Section	Contenu	Exemple
1	commandes utilisateur	<code>man 1 passwd</code> – la commande
5	formats de fichiers	<code>man 5 passwd</code> – le fichier /etc/passwd
8	administration système	<code>man 8 sudo</code>

Dans `man` : `/motif` pour chercher, `n` suivant, `q` pour quitter.

Réflexe professionnel : sur une instance EC2 **sans accès Internet sortant**, `man` fonctionne toujours. Savoir lire un manuel > mémoriser 200 options.

nano : l'éditeur du quotidien

```
nano notes.txt          # ouvrir (ou créer) un fichier
sudo nano /etc/hosts     # éditer un fichier système (droits root requis)
```

Tout est affiché en bas de l'écran —  signifie **Ctrl** :

Raccourci	Action
 +O puis 	enregistrer (O comme « Output »)
 +X	quitter (propose d'enregistrer si besoin)
 +W	rechercher dans le fichier
 +K /  +U	couper la ligne / coller
 +U	annuler la dernière modification

nano est **installé par défaut sur Ubuntu** et sera notre éditeur principal pour tout le cursus : fichiers de configuration, unités systemd (J2), scripts (J3), configuration SSH (J4).

vim : le kit de survie (vous le croiserez, c'est certain)

Sur certains systèmes minimaux, **seul vi/vim est disponible** (`visudo`, éditeur par défaut de git...). Il faut savoir en sortir sans débrancher la machine. vim a **deux modes** — toute la confusion vient de là :

```
vim essai.txt
```

Étape	Touche	Vous êtes en mode...
1. Entrer du texte	<code>i</code>	insertion (tapez normalement)
2. Revenir au mode commande	<code>Échap</code>	normal
3. Enregistrer et quitter	<code>:wq</code> puis <code>Entrée</code>	—
3 bis. Quitter SANS enregistrer	<code>:q!</code> puis <code>Entrée</code>	—

En cas de panique : `Échap` **plusieurs fois**, puis `:q!` — vous ressortez toujours indemne. C'est tout ce qu'on exige de vous sur vim dans ce cursus ; le reste (déplacements, `dd`, `/recherche`) viendra avec la pratique.

5. Utilisateurs, groupes, sudo

Qui a le droit de faire quoi

Où vivent les comptes : /etc/passwd et /etc/group

Chaque utilisateur = une ligne de `/etc/passwd` (lisible par tous) :

```
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
|      |      |      |      |      |      |
|      |      |      |      |      |      +-- shell de connexion
|      |      |      |      |      +-----+ répertoir HOME
|      |      |      |      +-----+ commentaire (nom complet)
|      |      |      +-----+ GID (groupe principal)
|      |      +-----+ UID (0 = root)
|      +-----+ mot de passe -> dans /etc/shadow
+-----+ login
```

- Les mots de passe **hachés** sont dans `/etc/shadow` (lisible par root seul).
- Les **groupes** et leurs membres : `/etc/group` (ex. `sudo:x:27:ubuntu`).

```
less /etc/passwd      # les comptes (beaucoup de comptes techniques : normal)
getent group sudo     # qui est dans le groupe sudo ?
```

Créer et modifier : adduser, usermod, id

```
sudo adduser alice          # créer alice : HOME, groupe, mot de passe (interactif)
sudo addgroup projet        # créer le groupe projet
sudo usermod -aG projet alice # Ajouter alice au Groupe projet
```

⚠ Le `-a` de `usermod -aG` est vital : **sans lui, vous remplacez** la liste des groupes de l'utilisateur au lieu de compléter. Classique et douloureux. Savoir qui on est et ce qu'on peut faire :

```
whoami          # mon login
id              # mes UID, GID et TOUS mes groupes
id alice        # les groupes d'alice
groups          # juste la liste des groupes
```

Note : les groupes sont lus **à la connexion** — alice devra se reconnecter (ou `su - alice`) pour que son nouveau groupe soit effectif.

root, su -, sudo : les trois façons d'être puissant

root (UID 0) a tous les droits. On ne travaille **jamais** connecté en root : une faute de frappe et plus de serveur.

```
su - alice      # devenir alice (session complète, mdp d'ALICE)
exit           # redevenir soi-même
sudo apt update # exécuter UNE commande en root (mdp de l'APPELANT)
sudo -i        # shell root complet - rare, à refermer vite
```

	su - alice	sudo commande
Mot de passe demandé	celui d'alice	le vôtre
Portée	toute une session	une commande
Traçabilité	faible	journalisée (qui, quand, quoi)

sudo est la voie normale d'administration — et **la seule sur EC2** :

le compte root n'a pas de mot de passe, on se connecte en **ubuntu** et on sudo.

Qui peut sudo ? Les membres du groupe **sudo** (`sudo usermod -aG sudo alice`).

6. Permissions

rwX : la serrure posée sur chaque fichier

Lire une ligne de droits

```
-rwxr-x--- 1 alice projet 4096 Jul  2 09:00 deploy.sh
|--|--|
| | | |
| | | +-- autres (o)      : --- rien
| | +----- groupe (g)   : r-x  lire, exécuter
| +----- propriétaire (u): rwx  lire, écrire, exécuter
+----- type : - fichier, d répertoire, l lien
```

Droit	Sur un fichier	Sur un répertoire
r	lire le contenu	lister (ls)
w	modifier	créer/supprimer dedans
x	exécuter	traverser (cd)

Trois blocs, toujours dans cet ordre : **u**ser, **g**roup, **o**thers.

chmod : symbolique pour ajuster, octal pour poser

Notation symbolique — qui (`u/g/o/a`), quoi (`+/-/=`), quel droit (`r/w/x`) :

```
chmod u+x deploy.sh      # le propriétaire peut exécuter (LE cas d'usage : scripts)
chmod g-w rapport.txt    # retirer l'écriture au groupe
chmod o= secret.txt      # les autres : plus aucun droit
```

Notation octale — chaque bloc devient un chiffre : `r=4, w=2, x=1`, on additionne :

Chiffre	Droits	Chiffre	Droits
7	rwX	4	r--
6	rw-	5	r-x
0	---	1	--X

```
chmod 640 rapport.txt    # u=rw-, g=r--, o=--- : fichier de données classique
chmod 750 deploy.sh      # u=rwx, g=r-x, o=--- : script d'équipe
chmod 600 cle-privee     # u=rw-, autres rien : le réflexe SSH (J4 !)
```

Symbolique = retoucher un droit ; octal = **poser l'état complet** d'un coup.

chown : changer de propriétaire et de groupe

Les droits ne suffisent pas : encore faut-il savoir **à qui** ils s'appliquent.

```
sudo chown alice rapport.txt      # changer le propriétaire
sudo chown alice:projet rapport.txt # propriétaire ET groupe d'un coup
sudo chown :projet rapport.txt    # le groupe seulement (ou chgrp projet)
sudo chown -R alice:projet /srv/app # -R : récursif sur toute l'arborescence
```

- **chown** est réservé à **root** (sinon on « donnerait » ses fichiers pour contourner les quotas et les droits).
- Vérification immédiate : `ls -l rapport.txt`.

Cas réel que vous vivrez en J4 : l'application StockLine tourne sous son propre utilisateur système, il faudra `chown -R stockline:stockline /opt/stockline` — un service qui ne peut pas écrire dans son répertoire de données, c'est le « Permission denied » le plus fréquent des mises en production.

umask : les droits par défaut des nouveaux fichiers

Qui décide des droits d'un fichier **au moment de sa création** ? L'**umask** : un masque qui **retire** des droits au maximum théorique.

```
maximum fichier : 666 (rw-rw-rw- : jamais x à la création)
umask           : 022
résultat       : 644 (rw-r--r--)

maximum répertoire : 777 (rwxrwxrwx)
umask             : 022
résultat         : 755 (rwxr-xr-x)
```

```
umask          # afficher le masque courant (format octal)
umask 027      # durcir pour la session : fichiers 640, répertoires 750
touch test.txt && ls -l test.txt # vérifier l'effet
```

- Les fichiers ne naissent **jamais exécutables** : d'où le `chmod u+x` sur vos scripts.
- Sur Ubuntu, l'umask par défaut de l'utilisateur est souvent `0002` (chaque utilisateur a son groupe privé) — vérifiez avec `umask`.

Le cas du répertoire : x = la clé du couloir

Sur un répertoire, les droits surprennent au début :

Droits sur le répertoire	Conséquence
<code>r--</code> sans <code>x</code>	<code>ls</code> marche (noms seuls), mais impossible d'entrer
<code>--x</code> sans <code>r</code>	<code>cd</code> marche, on accède aux fichiers si on connaît leur nom
<code>-wx</code>	on peut créer/supprimer dedans... y compris les fichiers des autres !

À bien intégrer :

- **Supprimer un fichier = droit `w` sur le RÉPERTOIRE**, pas sur le fichier :
un fichier en `400` dans un répertoire en `777` est supprimable par tous.
- Pour atteindre `/srv/partage/note.txt`, il faut `x` sur `/`, `/srv`
et `/srv/partage` : un seul maillon sans `x` et c'est « Permission denied ».
- Répertoire partagé d'équipe : `chmod 2770` — le `2` (**setgid**) force les nouveaux fichiers à hériter du **groupe du répertoire**. Vous le mettez en pratique tout de suite.



Exercice 1-2 – Droits d'un répertoire partagé

Fichier : `exercices/03-cl-linux/exercice-1-2-droits-repertoire-partage.md`

Durée : 45 min – difficulté 2/5 – seul, dans votre VM

Mise en situation : deux collègues doivent collaborer sur un même répertoire, **sans que personne d'autre n'y accède.**

- Créer les utilisateurs `alice` et `bruno`, le groupe `projet`.
- Créer `/srv/partage` accessible en écriture **au seul groupe** (`2770` + `setgid`).
- Vérifier avec `su - alice` puis `su - bruno` : chacun lit et modifie les fichiers de l'autre ; un troisième utilisateur est refusé.
- Observer l'effet du `setgid` et de `l'umask` sur les fichiers créés.
- Bonus : donner les droits `sudo` à `alice`.

Objectif : mobiliser **toute la chaîne** utilisateurs → groupes → permissions sur un cas que vous rencontrerez sur chaque serveur multi-utilisateurs.

Récap des points clés du jour

- **Linux** = l'OS du cloud ; un serveur s'administre au terminal, sans souris.
- **Noyau** = cœur commun ; **distribution** = noyau + outils + paquets ;
la nôtre : **Ubuntu Server 24.04 LTS**, celle de vos futures instances.
- VM : `multipass launch / shell / stop / start` – elle vit toute la semaine.
- Se repérer : `pwd`, `cd` (absolu vs relatif, `.` `..` `~`), `ls -lah`, `ls -ltr`.
- Manipuler : `cp -r`, `mv`, `mkdir -p`, et `rm -r` **sans corbeille ni retour**.
- Lire : `cat` (court), `less` (long), `head`/`tail`, `tail -f` (en direct).
- L'aide est sur la machine : `man commande`, sections, `--help`.
- Éditeurs : **nano** au quotidien ; vim : `i`, `Échap`, `:wq`, `:q!`.
- Comptes : `/etc/passwd` + `/etc/shadow`, `adduser`, `usermod -aG`,
`su -` vs `sudo` (tracé, une commande, la norme sur EC2).
- Permissions : `rwX` × u/g/o, octal (`chmod 640`), `chown`, `umask`,
et sur un répertoire : **x** = **traverser**, **w** = **créer/supprimer dedans**.



Quiz — questions 1 et 2

Question 1

Quelle est la différence entre le noyau Linux et une distribution ?

Question 2

Que fait la commande `cd` lancée sans argument ?



Quiz — questions 3 et 4

Question 3

Dans un chemin, que désignent `.` et `..` ?

Question 4

Quelle commande affiche le manuel complet de `ls` ?



Quiz — questions 5 et 6

Question 5

Que fait `rm -r projets/` et pourquoi faut-il s'en méfier ?

Question 6

Dans `-rwxr-x---`, quels sont les droits du groupe ?



Quiz — questions 7 et 8

Question 7

Que fait `chmod 640 rapport.txt` ?

Question 8

Quelle est la différence entre `su - alice` et `sudo` ?



Quiz — questions 9 et 10

Question 9

Dans quel fichier les utilisateurs locaux sont-ils déclarés ?

Question 10

Avec `umask 022`, quels droits reçoit un fichier nouvellement créé ?

À demain !

Jour 2 — Faire tourner des services

Vous savez maintenant **vous déplacer et donner des droits**.

Demain, la machine s'anime : **processus** (ps, top, kill), **services systemd**

(vous écrirez votre première unité), paquets APT et lecture des **journaux** —

tout ce qu'il faut pour qu'une application tourne **sans vous**.

Ne supprimez pas votre VM : `multipass stop srv-linux` suffit.